Introduction to Gradient-Based MCMC Samplers

EDUC 231A Guest Lecture

Tianying (Teanna) Feng

2026-05-21

Outline

- Start with a target distribution we want to *explore* and *sample from*
- Review the Metropolis-Hastings (MH) sampler
- Introduce three gradient-based MCMC samplers
  - Metropolis-Adjusted Langevin Algorithm (MALA)
  - Hamiltonian Monte Carlo (HMC)
  - No-U-Turn Sampler (NUTS)

The target distribution

Multivariate Gaussian distribution

For $\mathbf{x} \in \mathbb{R}^d$ with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$,

$$p(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

The log-likelihood is

$$\log p(\mathbf{x}) = -\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) - \tfrac{d}{2}\log(2\pi) - \tfrac{1}{2}\log|\boldsymbol{\Sigma}|$$

The potential $U(\mathbf{x})$

We will be working with $U(\mathbf{x}) = -\log p(\mathbf{x}) + \text{const}$, the negative log-likelihood up to a constant.

The constant term (w.r.t $\mathbf{x}$) will cancel in the accept/reject ratio, and thus we focus on

$$U(\mathbf{x}) = \tfrac{1}{2}(\mathbf{x} - \boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

$$\nabla U(\mathbf{x}) = \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})$$

Deriving the gradient $\nabla U(\mathbf{x})$

Start from the full log-likelihood:

$$\log p(\mathbf{x}) = -\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})'\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) \underbrace{-\tfrac{d}{2}\log(2\pi) - \tfrac{1}{2}\log|\boldsymbol{\Sigma}|}_{\text{constant in } \mathbf{x}}$$

Write $\mathbf{a} = \mathbf{x} - \boldsymbol{\mu}$, so $U = \tfrac{1}{2}\mathbf{a}'\boldsymbol{\Sigma}^{-1}\mathbf{a}$ up to that constant. For a symmetric matrix $\mathbf{M}$,

$$\tfrac{1}{2}\nabla_{\mathbf{a}}\left(\mathbf{a}'\mathbf{M}\,\mathbf{a}\right) = \tfrac{1}{2}(\mathbf{M}+\mathbf{M}')\,\mathbf{a}$$

With $\mathbf{M} = \boldsymbol{\Sigma}^{-1}$ (and $\boldsymbol{\Sigma}$ is symmetric) and $\nabla_{\mathbf{x}}\mathbf{a} = \mathbf{I}$, the chain rule gives

$$\nabla U(\mathbf{x}) = \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})$$

Bivariate Gaussian with correlation ρ

Take $\boldsymbol{\mu} = \mathbf{0}$ and

$$\boldsymbol{\Sigma} = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \qquad \boldsymbol{\Sigma}^{-1} = \frac{1}{1-\rho^2} \begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix}$$

then

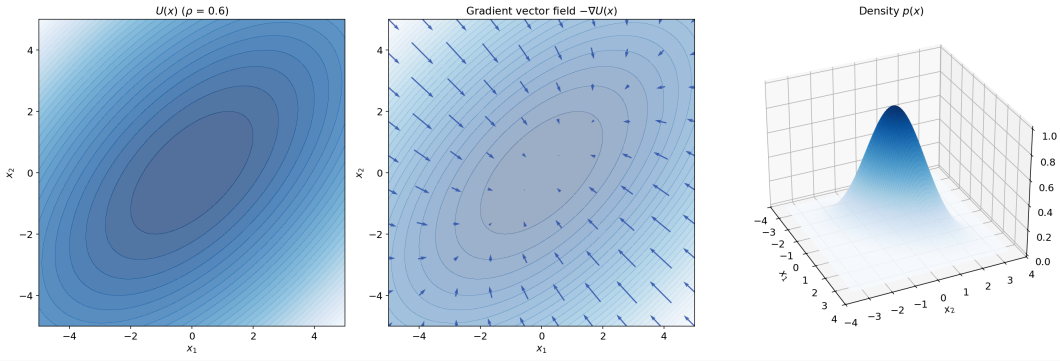
$$U(x_1, x_2) = \frac{x_1^2 - 2\rho\, x_1 x_2 + x_2^2}{2(1-\rho^2)}$$

$$\nabla U(x_1, x_2) = \frac{1}{1-\rho^2} \begin{pmatrix} x_1 - \rho\, x_2 \\ x_2 - \rho\, x_1 \end{pmatrix}$$

Define the target distribution in code

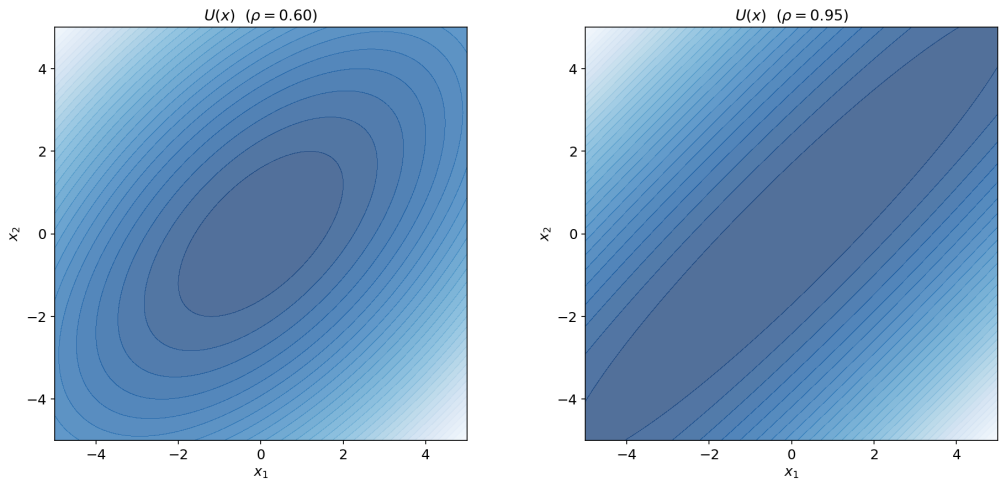
```
def U(x):
    return 0.5 * x @ Sigma_inv @ x

def grad_U(x):
    return Sigma_inv @ x
```

Visualize the target distribution



Left: the potential U. Middle: the field $-\nabla U$ points toward the mode. Right: the density $p(\mathbf{x})$ itself.

What happens when ρ is large?



Metropolis–Hastings (MH)

MH: proposal

A symmetric Gaussian centered at the current state:

$$q(\mathbf{x}' \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}' \mid \mathbf{x}, \sigma^2 \mathbf{I}) \tag{1}$$

$$\mathbf{x}' = \mathbf{x} + \sigma\, \boldsymbol{\eta} \tag{2}$$

where $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

MH: accept / reject

The Metropolis–Hastings ratio is

$$\alpha(\mathbf{x}, \mathbf{x}') = \min\left(1, \frac{p(\mathbf{x}')\, q(\mathbf{x} \mid \mathbf{x}')}{p(\mathbf{x})\, q(\mathbf{x}' \mid \mathbf{x})}\right) \tag{3}$$

For a symmetric proposal $q(\mathbf{x}' \mid \mathbf{x}) = q(\mathbf{x} \mid \mathbf{x}')$, the q-ratio cancels:

$$\alpha(\mathbf{x}, \mathbf{x}') = \min\left(1,\ e^{-(U(\mathbf{x}') - U(\mathbf{x}))}\right) \tag{4}$$

Alternatively

Accept iff $\log u < -(U(\mathbf{x}') - U(\mathbf{x}))$ with $u \sim \text{Uniform}(0, 1)$.

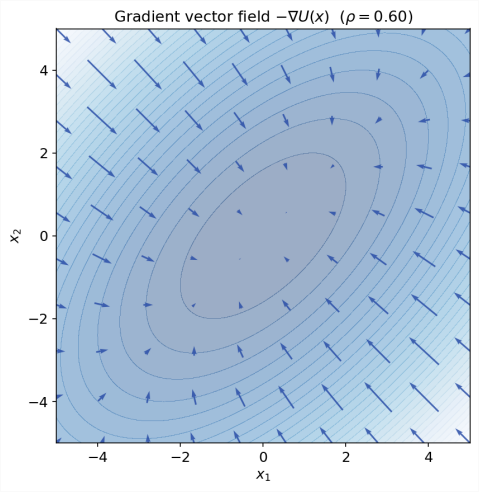
MH: implementation

```
def mh_step(x, U, sigma, rng):
    x_new = x + sigma * rng.standard_normal(x.shape)
    log_alpha = -(U(x_new) - U(x))
    accepted = np.log(rng.random()) < log_alpha
    return (
        x_new if accepted else x,
        accepted,
        x_new
    )
```

MH: demo time

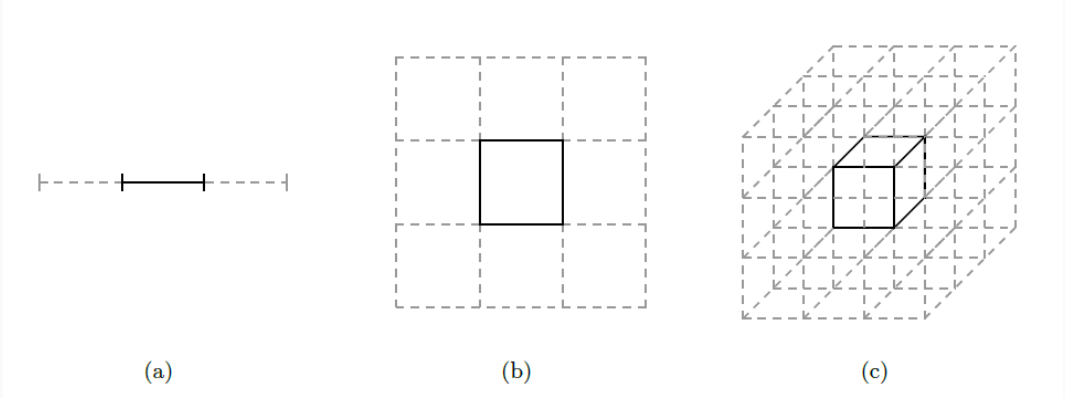
Let's see it in action

MH: random-walk behavior ignores the gradient



The arrows show $-\nabla U(\mathbf{x})$: at every point there is an exact direction toward higher density.

MH: why random walk gets inefficient in high dimensions?



Metropolis-Adjusted Langevin Algorithm (MALA)

MALA: proposal

A Gaussian centered at the gradient-drifted mean:

$$\mu_{\text{fwd}}(\mathbf{x}) = \mathbf{x} - \tfrac{\varepsilon^2}{2}\,\nabla U(\mathbf{x}) \tag{5}$$

$$q(\mathbf{x}' \mid \mathbf{x}) = \mathcal{N}(\mathbf{x}' \mid \mu_{\text{fwd}}(\mathbf{x}),\, \varepsilon^2 \mathbf{I}) \tag{6}$$

$$\mathbf{x}' = \mu_{\text{fwd}}(\mathbf{x}) + \varepsilon\,\boldsymbol{\eta} \tag{7}$$

Key idea

The drift term $-\frac{\varepsilon^2}{2}\nabla U(\mathbf{x})$ moves toward higher density.

MALA: the proposal density

The forward density is Gaussian with the forward-drift mean above:

$$\log q(\mathbf{x}' \mid \mathbf{x}) = -\tfrac{1}{2\varepsilon^2} \left\| \mathbf{x}' - \mu_{\text{fwd}}(\mathbf{x}) \right\|^2 + \text{const} \tag{8}$$

The reverse move $\mathbf{x}' \to \mathbf{x}$ drifts from $\mathbf{x}'$, using the backward-drift mean:

$$\mu_{\text{bwd}}(\mathbf{x}') = \mathbf{x}' - \tfrac{\varepsilon^2}{2} \nabla U(\mathbf{x}') \tag{9}$$

$$\log q(\mathbf{x} \mid \mathbf{x}') = -\tfrac{1}{2\varepsilon^2} \left\| \mathbf{x} - \mu_{\text{bwd}}(\mathbf{x}') \right\|^2 + \text{const} \tag{10}$$

Why it is asymmetric

$\mu_{\text{fwd}}(\mathbf{x}) \neq \mu_{\text{bwd}}(\mathbf{x}')$ because each drift uses the gradient at its own start point, in which case $q(\mathbf{x}' \mid \mathbf{x}) \neq q(\mathbf{x} \mid \mathbf{x}')$ and the q-ratio does *not* cancel.

MALA: accept / reject

The Metropolis–Hastings ratio is

$$\alpha(\mathbf{x}, \mathbf{x}') = \min\left(1, \frac{p(\mathbf{x}')\, q(\mathbf{x} \mid \mathbf{x}')}{p(\mathbf{x})\, q(\mathbf{x}' \mid \mathbf{x})}\right) \tag{11}$$

Substituting the two proposal densities from the previous slide and using $U = -\log p + \text{const}$:

$$\alpha(\mathbf{x}, \mathbf{x}') = \min\left(1,\ e^{-(U(\mathbf{x}') - U(\mathbf{x})) + \log q(\mathbf{x}|\mathbf{x}') - \log q(\mathbf{x}'|\mathbf{x})}\right) \tag{12}$$

Contrast with MH

The extra $\log q$ terms are the asymmetry correction. They cancel out for a symmetric proposal.

MALA: implementation

```
def mala_step(x, U, grad_U, eps, rng):
    g = grad_U(x)
    mu_fwd = x - 0.5 * eps**2 * g
    x_new = mu_fwd + eps * rng.standard_normal(x.shape)

    g_new = grad_U(x_new)
    mu_bwd = x_new - 0.5 * eps**2 * g_new

    log_q_fwd = -0.5 * np.sum((x_new - mu_fwd)**2) / eps**2
    log_q_bwd = -0.5 * np.sum((x - mu_bwd)**2) / eps**2

    log_alpha = -(U(x_new) - U(x)) + (log_q_bwd - log_q_fwd)
    accepted = np.log(rng.random()) < log_alpha
    return (x_new if accepted else x, accepted, x_new)
```

MALA: demo time

Let's see it in action

Hamiltonian Monte Carlo (HMC)

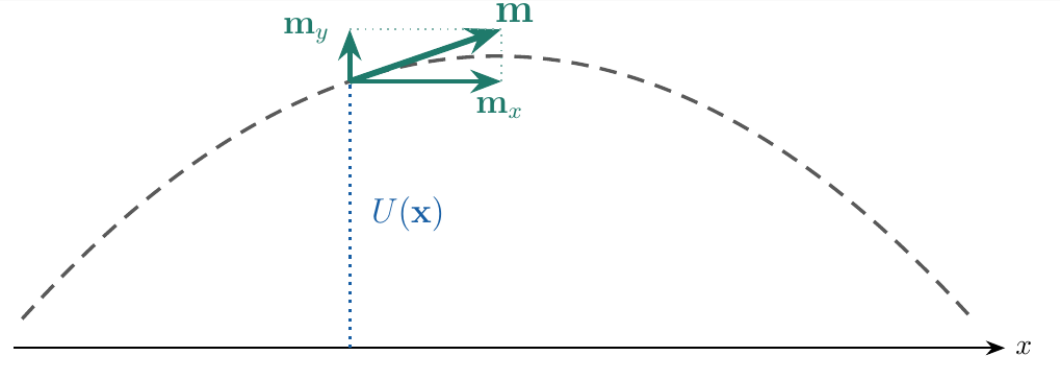
HMC: Hamiltonian dynamics

The Hamiltonian is a potential plus a kinetic energy $H(\mathbf{x}, \mathbf{m}) = U(\mathbf{x}) + K(\mathbf{m})$

The "frictionless puck" analogy from Neal (2011)

"[W]e can visualize the dynamics as that of a frictionless puck that slides over a surface of varying height. The state of this system consists of the position of the puck ... and the momentum of the puck (its mass times its velocity) ... The potential energy, $U(q)$, of the puck is proportional to the height of the surface at its current position, and its kinetic energy, $K(p)$, is ... [related to] ... the mass of the puck."

HMC: Hamiltonian dynamics



HMC: Hamiltonian dynamics

Let $K(\mathbf{m}) = \frac{1}{2}\|\mathbf{m}\|^2$, then

$$H(\mathbf{x}, \mathbf{m}) = U(\mathbf{x}) + \frac{1}{2}\|\mathbf{m}\|^2$$
$$\frac{\partial H}{\partial \mathbf{m}} = \mathbf{m}$$
$$\frac{\partial H}{\partial \mathbf{x}} = \nabla U(\mathbf{x})$$

Furthermore,

$$\frac{d\mathbf{x}}{dt} = \frac{\partial H}{\partial \mathbf{m}} = \mathbf{m}$$
$$\frac{d\mathbf{m}}{dt} = -\frac{\partial H}{\partial \mathbf{x}} = -\nabla U(\mathbf{x})$$

HMC: the leapfrog integrator

Integrate with step size ε for $l = 1, \ldots, L$: a half-step in momentum, a full step in position, then a second half-step in momentum.

The general equations are

$$\begin{aligned}
\mathbf{m}_{l+1/2} &= \mathbf{m}_l - \tfrac{\varepsilon}{2} \tfrac{\partial H}{\partial \mathbf{x}}(\mathbf{x}_l) \\
\mathbf{x}_{l+1} &= \mathbf{x}_l + \varepsilon \tfrac{\partial H}{\partial \mathbf{m}}(\mathbf{m}_{l+1/2}) \\
\mathbf{m}_{l+1} &= \mathbf{m}_{l+1/2} - \tfrac{\varepsilon}{2} \tfrac{\partial H}{\partial \mathbf{x}}(\mathbf{x}_{l+1})
\end{aligned}$$

HMC: the leapfrog integrator

For this H ($\partial H/\partial \mathbf{x} = \nabla U$, $\partial H/\partial \mathbf{m} = \mathbf{m}$), the leapfrog steps become

$$\mathbf{m}_{l+1/2} = \mathbf{m}_l - \tfrac{\varepsilon}{2} \nabla U(\mathbf{x}_l)$$

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \varepsilon\, \mathbf{m}_{l+1/2}$$

$$\mathbf{m}_{l+1} = \mathbf{m}_{l+1/2} - \tfrac{\varepsilon}{2} \nabla U(\mathbf{x}_{l+1})$$

HMC: implementation

```
def leapfrog(x, m, grad_U, eps, L):
    x, m = x.copy(), m.copy()
    traj = [x.copy()]
    for l in range(L):
        m -= 0.5 * eps * grad_U(x)   # half step in m
        x += eps * m                 # full step in x
        m -= 0.5 * eps * grad_U(x)   # half step in m
        traj.append(x.copy())
    return x, m, traj
```

HMC: proposal

At each iteration,

1. Sample the momentum independent of $\mathbf{x}$: $\mathbf{m} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. Follow the dynamics from $(\mathbf{x}, \mathbf{m})$ for a fixed time
3. The **proposal** is the end of the trajectory $(\mathbf{x}', \mathbf{m}')$
4. The MH accept ratio corrects the small drift from finite ε

$$\alpha = \min\left(1,\ e^{-(H(\mathbf{x}',\mathbf{m}')-H(\mathbf{x},\mathbf{m}))}\right)$$

HMC: implementation

```
def hmc_step(x, U, grad_U, eps, L, rng):
    m = rng.standard_normal(x.shape)
    H0 = U(x) + 0.5 * np.sum(m**2)
    x_new, m_new, traj = leapfrog(x, m, grad_U, eps, L)
    H1 = U(x_new) + 0.5 * np.sum(m_new**2)
    log_alpha = -(H1 - H0)
    accepted = np.log(rng.random()) < log_alpha
    return (x_new if accepted else x, accepted, x_new, traj)
```

HMC: demo time

Let's see it in action

No-U-Turn Sampler (NUTS)

NUTS: motivation

HMC requires manual turning of the trajectory length L:

- Too small: short moves that start to mimic random-walk behavior
- Too large: the trajectory loops back

Key idea

Choose the trajectory length *automatically*, per iteration, instead of fixing one L for the whole run.

NUTS: how a U-turn is detected?

We stop when the (half) squared distance between the fixed start $\mathbf{x}^-$ and the moving end $\mathbf{x}^+$ does not increase anymore:

$$\underbrace{\frac{d}{dt}\left[\tfrac{1}{2}\|\mathbf{x}^+ - \mathbf{x}^-\|^2\right]}_{\text{rate of change of squared distance}} = (\mathbf{x}^+ - \mathbf{x}^-) \cdot \underbrace{\left(\frac{d\mathbf{x}^+}{dt} - \frac{d\mathbf{x}^-}{dt}\right)}_{\mathbf{m}^+}$$

$$= (\mathbf{x}^+ - \mathbf{x}^-) \cdot \mathbf{m}^+ \qquad \mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\|\,\|\mathbf{b}\| \cos\theta$$

$$= \|\mathbf{x}^+ - \mathbf{x}^-\|\,\|\mathbf{m}^+\| \cos\theta$$

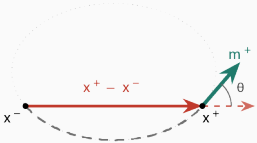
Both norms are positive, so the sign is governed entirely by $\cos\theta$. A U-turn is detected when

$$(\mathbf{x}^+ - \mathbf{x}^-) \cdot \mathbf{m}^+ < 0 \qquad \Longleftrightarrow \qquad \theta > 90°$$

NUTS: how a U-turn is detected?

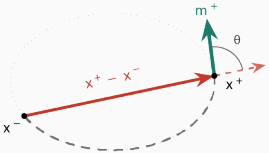
As the trajectory bends, $\mathbf{m}^+$ rotates from pointing *away* from $\mathbf{x}^-$ ($\theta < 90°$) to pointing *back* toward it ($\theta > 90°$).

1. Still growing



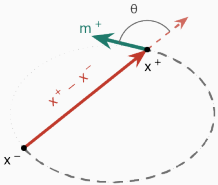
$(x^+ - x^-) \cdot m^+ > 0$

2. Turning point



$(x^+ - x^-) \cdot m^+ \approx 0$

3. U-turn detected



$(x^+ - x^-) \cdot m^+ < 0$

NUTS: in practice

- No trajectory length L to tune compared to HMC
- The step size ε can be adapted during warm-up iterations
- Advanced version implemented in Stan, PyMC, and NumPyro

Questions?

References

Betancourt, M. (2018).**A conceptual introduction to Hamiltonian Monte Carlo.** *arXiv*. https://doi.org/https://doi.org/10.48550/arXiv.1701.02434

Hoffman, M. D., & Gelman, A. (2014).**The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo.** *Journal of Machine Learning Research*, *15*, 1593–1623. https://doi.org/https://doi.org/10.48550/arXiv.1111.4246

Neal, R. M. (2011). **MCMC using Hamiltonian dynamics.** In S. Brooks, A. Gelman, G. Jones, & X.-L. Meng (Eds.), *Handbook of Markov Chain Monte Carlo*. Chapman & Hall / CRC Press. https://doi.org/https://doi.org/10.1201/b10905

Xifara, T., Sherlock, C., Livingstone, S., Byrne, S., & Girolami, M. (2014).**Langevin diffusions and the Metropolis-adjusted Langevin algorithm.** *Statistics & Probability Letters*, *91*, 14–19. https://doi.org/10.1016/j.spl.2014.04.002